AgentLens — Complete Build Plan v2 (with Intervention & Recovery)

Build India Hackathon | Feb 15, 2026 | 7 Hours | Solo

---

1. WHAT IS THIS NOW?

Not just a debugger. An **agent immune system**.

It intercepts every decision an AI agent makes, detects failure patterns in real-time, **automatically intervenes to prevent failures before they compound**, and if failures do happen, lets you rewind to a healthy checkpoint and resume with a corrected strategy.

One-liner pitch: "An immune system for AI agents — detects, prevents, and heals agent failures in real-time."

Why "immune system" framing is better than "debugger":

- Debugger = passive, post-hoc, developer-only tool
- Immune system = active, real-time, works automatically
- Judges hear "debugger" and think "nice to have"
- Judges hear "immune system that self-heals agents" and think "everyone needs this"

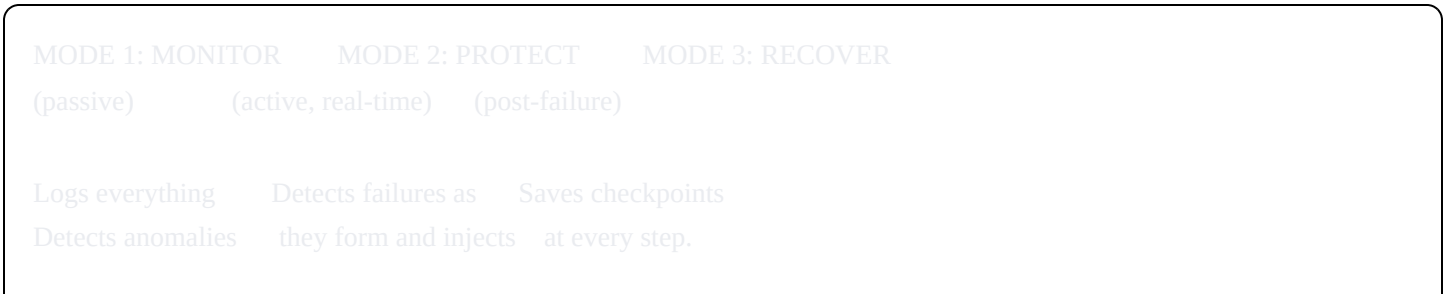
The problem (what you tell judges):

"Every team deploying AI agents hits the same wall. The agent starts great, then 20 minutes in it starts looping, contradicting itself, burning through context, and spiraling. Today you have two options: stare at the chat log trying to figure out what happened, or kill the session and start over.

AgentLens is the third option. It watches your agent in real-time, detects failures as they form, automatically intervenes to course-correct, and if things go too far wrong, lets you rewind to a healthy checkpoint and resume with a fix applied. It's an immune system for AI agents."

---

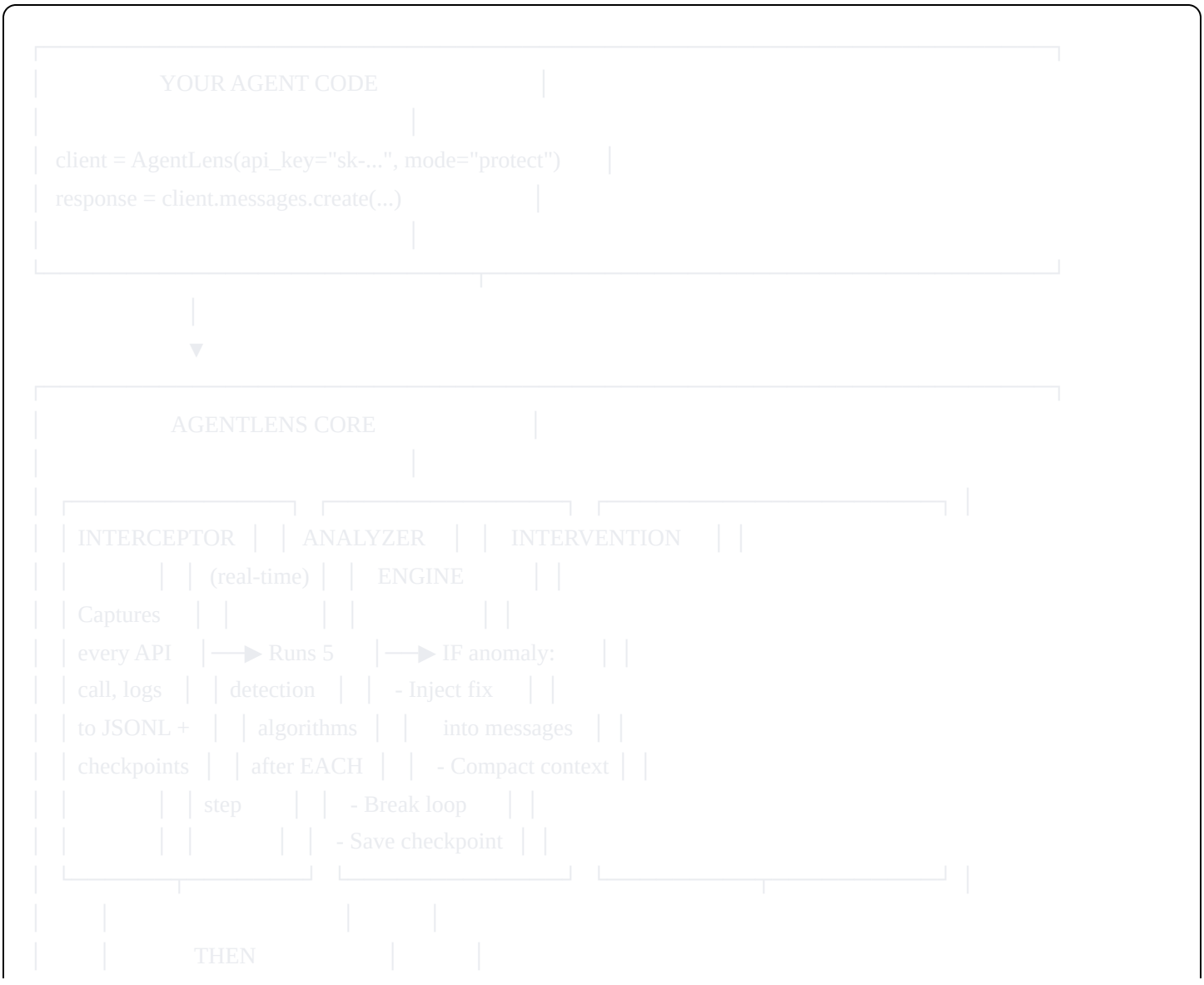
2. THE THREE MODES OF OPERATION

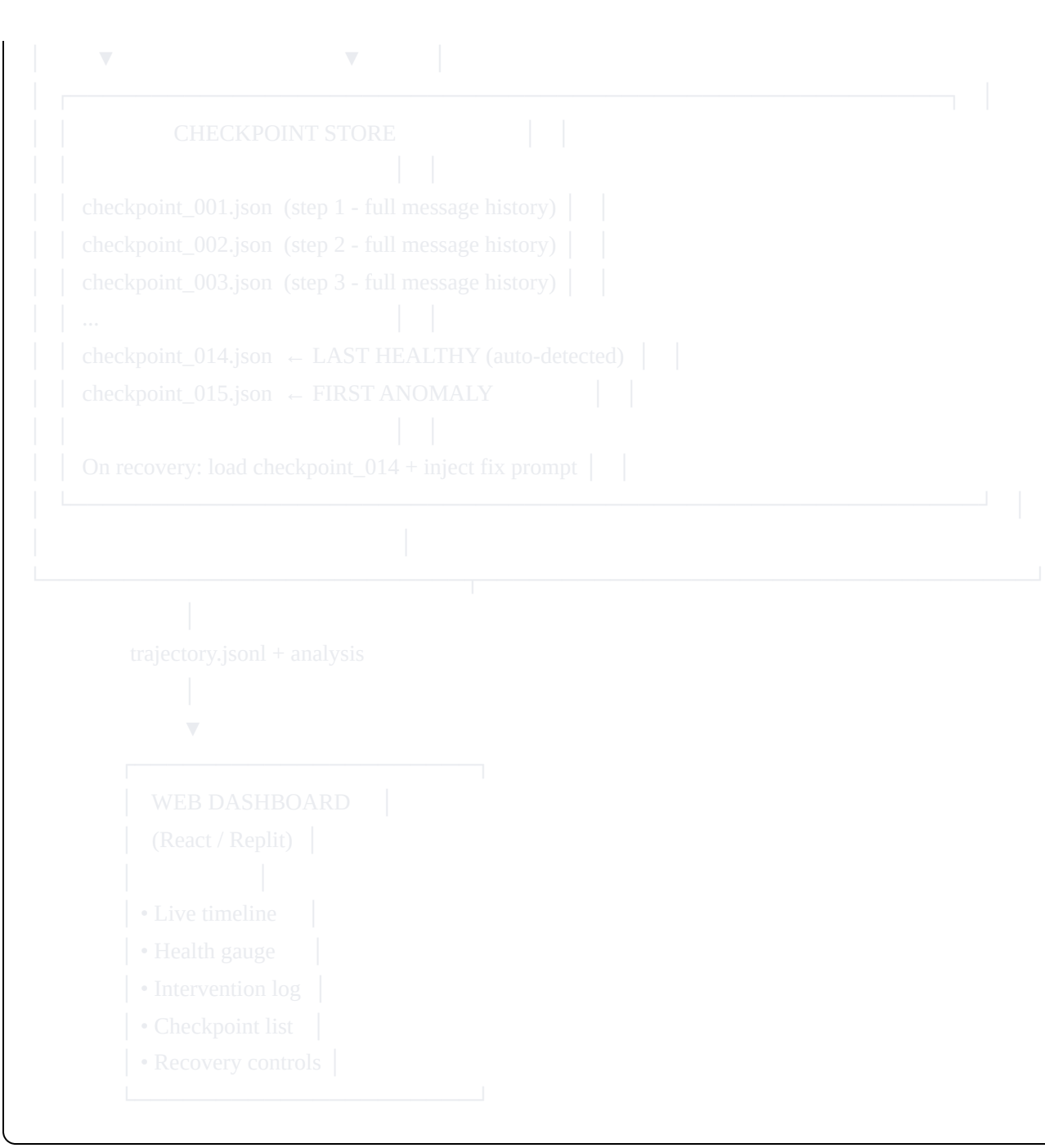


The demo shows all three: "Here's what happened (Monitor) → Here's how we prevent it (Protect) → And if it still fails, here's how we recover (Recover)."

---

3. UPDATED ARCHITECTURE





---

4. THE INTERVENTION ENGINE — HOW IT WORKS

This is the new core feature. When the analyzer detects an anomaly between API calls, the intervention engine modifies the NEXT request before it goes to Claude.

4a. Loop Breaking

Detection: Same sequence of tool calls repeated 2+ times.

Intervention: Inject a system-level message into the conversation:

```
python
```

```
intervention_message = {
    "role": "user",
    "content": (
        "[SYSTEM NOTICE] You appear to be in a loop. You have called "
        f"{tool_name} on {file_path} {count} times in the last {window} steps "
        "with the same or similar inputs. STOP and reconsider your approach. "
        "Summarize what you've tried so far, what failed, and propose a "
        "fundamentally different strategy before making any more tool calls."
    )
}
# Inject this before the next API call
```

Why this works: LLMs respond strongly to explicit "you are looping" signals. The key is that a human never has to type this — AgentLens injects it automatically. The agent course-corrects and the user sees a smooth session.

4b. Context Compaction Trigger

Detection: Context utilization > 70% AND growing at > 3% per step.

Intervention: Before the next API call, insert a compaction request:

```
python

compaction_message = {
    "role": "user",
    "content": (
        "[SYSTEM NOTICE] Your context window is {pct}% full. Before proceeding, "
        "create a concise summary of: (1) what you've accomplished so far, "
        "(2) what remains to be done, (3) key decisions made and their rationale. "
        "Then continue with your task using this summary as your reference."
    )
}
```

Additionally: Trim older messages from the conversation history, keeping only the system prompt + last N messages + the compaction summary. This is programmatic context management — the interceptor actually modifies the messages array before sending it to the API.

4c. Error Spiral Breaker

Detection: 3+ consecutive steps with errors.

Intervention:

```
python
```

```
error_breaker_message = {
    "role": "user",
    "content": (
        "[SYSTEM NOTICE] You have encountered {n} consecutive errors. "
        "The error-fix-error cycle is not working. STOP making changes. "
        "Instead: (1) Read the full error message carefully. "
        "(2) Check if your fundamental approach is wrong, not just the syntax. "
        "(3) Consider reverting your last {n} changes and trying a completely "
        "different approach. (4) If the error is in a dependency or environment "
        "issue, acknowledge it rather than trying to fix it in code."
    )
}
```

4d. Repetition Reducer

Detection: Same file read 3+ times without a write in between.

Intervention:

python

```
repetition_message = {
    "role": "user",
    "content": (
        "[SYSTEM NOTICE] You have read {file_path} {count} times without "
        "making meaningful changes. You appear to already have this information. "
        "Do NOT read this file again. Work with the information you have."
    )
}
```

4e. Strategy Drift Corrector (uses Claude meta-analysis)

Detection: Every 10 steps, run a lightweight meta-check.

Intervention: Send a summary of the last 10 steps to Claude (separate API call) and ask "Is the agent still on track with its original plan?" If drift detected:

python

```
drift_message = {
    "role": "user",
    "content": (
        "[SYSTEM NOTICE] You appear to have drifted from your original plan. "
        "Your original goal was: {original_task}. "
        "In the last 10 steps, you've been: {drift_summary}. "
        "Realign with the original objective before continuing."
    )
}
```

---

5. THE CHECKPOINT + RECOVERY SYSTEM

How Checkpoints Work:

Every step, BEFORE the API call, save:

python

```
checkpoint = {
    "step_id": N,
    "messages": copy.deepcopy(current_messages),  # Full conversation state
    "tools": current_tools,                       # Tool configuration
    "system_prompt": current_system_prompt,
    "metadata": {
        "health_score": current_health,
        "context_utilization": current_pct,
        "anomalies_so_far": [],
        "timestamp": datetime.utcnow().isoformat()
    }
}
# Save to checkpoints/step_{N}.json
```

Storage: Just JSON files in a `checkpoints/` directory. For a 30-step session, that's ~30 JSON files. Small, fast, reliable.

How Recovery Works:

When a failure is detected (or user triggers manually from dashboard):

Step 1: Identify the LAST HEALTHY checkpoint

python

```python
def find_recovery_point(checkpoints, anomalies):
    """Find the last step before any anomaly started"""
    first_anomaly_step = min(a["step_id"] for a in anomalies)
    # Go back 2 steps from first anomaly for safety margin
    recovery_step = max(1, first_anomaly_step - 2)
    return checkpoints[recovery_step]
```

Step 2: Generate a FIX STRATEGY using Claude meta-analysis

```python
def generate_fix_strategy(trajectory_so_far, anomalies):
    """Ask Claude to analyze what went wrong and how to avoid it"""
    client = anthropic.Anthropic()
    response = client.messages.create(
        model="claude-sonnet-4-5-20250929",
        max_tokens=800,
        messages=[{
            "role": "user",
            "content": f"""Analyze this agent trajectory that failed:

Steps taken: {json.dumps(trajectory_so_far)}
Failures detected: {json.dumps(anomalies)}

Generate a CONCISE correction prompt (max 3 sentences) that:
1. Warns the agent about the specific failure pattern it fell into
2. Suggests a concrete alternative approach
3. Sets a constraint to prevent the same failure

Return ONLY the correction prompt text, nothing else."""
        }]
    )
    return response.content[0].text
```

Step 3: Load checkpoint + inject fix + resume

```python
```

```
def recover(checkpoint, fix_strategy):
    """Resume agent from healthy checkpoint with fix applied"""
    messages = checkpoint["messages"]

    # Inject the fix strategy as a system-level message
    messages.append({
        "role": "user",
        "content": f"[RECOVERY] Previous approach failed. {fix_strategy} "
                   f"Continue from where you left off with this corrected approach."
    })

    # Resume the agent with corrected state
    return messages
```

What the user sees in the dashboard:

```
Timeline: [1]🟢 [2]🟢 ... [12]🟢 [13]🟡 [14]🔴 [15]🔴 ...

                    ↑
              RECOVERY POINT
            "Step 12 was healthy"

[Button: "Recover from Step 12"]

→ Clicking this:
  1. Shows the fix strategy Claude generated
  2. User can edit the fix or accept
  3. Resumes agent from step 12 with fix injected
  4. New timeline branch appears: [12]🟢 → [12a]🟢 [13a]🟢 ...
```

---

6. MVP FEATURES — WHAT MUST WORK AT SUBMISSION

MUST HAVE (Non-negotiable):

1. **Instrumented SDK Wrapper** — Drop-in replacement for anthropic.Anthropic()
   - Logs every call to JSONL
   - Saves checkpoints at every step
   - Runs anomaly detection after each step (real-time)
2. **Real-Time Intervention** (at least 3 of 5 interventions):

- ✅ Loop breaking (inject course-correction prompt)
- ✅ Context compaction trigger (inject summary request + trim messages)
- ✅ Error spiral breaker (inject "stop and rethink" prompt)
- ⬜ Repetition reducer (nice to have)
- ⬜ Strategy drift corrector (nice to have — needs extra API call)

3. **Checkpoint + Recovery**:
   - ✅ Save checkpoints at every step
   - ✅ Identify last healthy checkpoint
   - ✅ Generate fix strategy via Claude
   - ✅ Resume from checkpoint with fix injected
4. **Web Dashboard** with:
   - ✅ Timeline showing step health + anomaly markers
   - ✅ Health score gauge
   - ✅ Context budget chart
   - ✅ Intervention log (shows WHAT AgentLens did and WHEN)
   - ✅ Recovery panel (shows checkpoint list + recovery button)
   - ✅ Step detail on click
5. **Compelling Demo Trajectory**:
   - ✅ A trajectory showing an agent failing WITHOUT AgentLens
   - ✅ Same task WITH AgentLens showing interventions saving the session

NICE TO HAVE (if time permits):

- Claude meta-analysis narrative ("here's what went wrong in plain English")
- Side-by-side comparison: unprotected vs protected trajectory
- Live websocket streaming (dashboard updates in real-time as agent runs)
- Tool distribution chart
- Export trajectory as shareable report

---

7. TECH STACK (unchanged)

```
PYTHON SDK (interceptor + analyzer + interventions + checkpoints):
├── Python 3.11+
```

```
├── anthropic SDK
├── dataclasses + json
├── copy (for deep-copying message state)
├── hashlib (for tool call hashing)
└── pathlib (for checkpoint file management)

WEB DASHBOARD (React on Replit):
├── React 18
├── Tailwind CSS
├── Recharts (charts)
├── Lucide React (icons)
└── Deployed on Replit (public URL)

NO external dependencies beyond these. No databases. No Redis.
No Docker. No websockets (use file upload for MVP).
```

---

8. UPDATED HOUR-BY-HOUR BUILD PLAN

HOUR 0–0:45 — Interceptor + Checkpoint System

Goal: Working SDK wrapper that logs + saves checkpoints

Build `agentlens/interceptor.py`:

```python
```

```python
import anthropic
import json
import copy
import time
import os
from datetime import datetime
from pathlib import Path


class AgentLens:
    """
    Drop-in replacement for anthropic.Anthropic() that adds:
    - Full trajectory logging (JSONL)
    - Checkpoint saving at every step
    - Real-time anomaly detection
    - Automatic intervention when failures detected
    """

    def __init__(self, mode="protect", checkpoint_dir="checkpoints",
                 log_file="trajectory.jsonl", **kwargs):
        self._client = anthropic.Anthropic(**kwargs)
        self._mode = mode  # "monitor" | "protect" | "recover"
        self._checkpoint_dir = Path(checkpoint_dir)
        self._checkpoint_dir.mkdir(exist_ok=True)
        self._log_file = open(log_file, "a")
        self._step = 0
        self._cumulative_input = 0
        self._cumulative_output = 0
        self._tool_history = []       # List of (tool_name, input_hash)
        self._error_streak = 0
        self._file_read_counts = {}   # file_path -> count since last write
        self._interventions = []      # Log of all interventions made
        self._anomalies = []

        self.messages = self  # Allows client.messages.create() pattern

    def create(self, **kwargs):
        self._step += 1
        messages = kwargs.get("messages", [])

        # ── CHECKPOINT: save state BEFORE the call ──
        self._save_checkpoint(kwargs)
```

```
    # ── ANALYZE: check for anomalies from previous steps ──
    if self._mode == "protect" and self._step > 3:
        intervention = self._check_and_intervene(kwargs)
        if intervention:
            # Modify the messages before sending
            kwargs["messages"] = intervention["modified_messages"]
            self._interventions.append(intervention)

    # ── CALL: make the actual API request ──
    start = time.time()
    response = self._client.messages.create(**kwargs)
    duration = (time.time() - start) * 1000

    # ── LOG: capture everything ──
    log_entry = self._build_log_entry(kwargs, response, duration)
    self._log_file.write(json.dumps(log_entry) + "\n")
    self._log_file.flush()

    # ── UPDATE: internal tracking state ──
    self._update_tracking(response, log_entry)

    return response

def _save_checkpoint(self, kwargs):
    checkpoint = {
        "step_id": self._step,
        "timestamp": datetime.utcnow().isoformat(),
        "messages": copy.deepcopy(kwargs.get("messages", [])),
        "model": kwargs.get("model"),
        "system": kwargs.get("system", ""),
        "tools": [t.get("name", "") if isinstance(t, dict)
                  else str(t) for t in kwargs.get("tools", [])],
        "health_snapshot": {
            "context_pct": self._context_pct(),
            "error_streak": self._error_streak,
            "anomaly_count": len(self._anomalies),
        }
    }
    path = self._checkpoint_dir / f"step_{self._step:04d}.json"
    with open(path, "w") as f:
        json.dump(checkpoint, f)

def _check_and_intervene(self, kwargs):
    """Run anomaly detection and return intervention if needed."""
```

```
        messages = kwargs.get("messages", [])

        # CHECK 1: Loop detection
        loop = self._detect_loop()
        if loop:
            self._anomalies.append({
                "type": "LOOP", "step": self._step, "severity": "HIGH",
                "detail": loop
            })
            fix_msg = (
                f"[AGENTLENS] Loop detected: you've repeated the pattern "
                f"'{loop['pattern']}' {loop['count']} times. STOP. "
                f"Summarize what you've tried, explain why it's not working, "
                f"and propose a fundamentally different approach."
            )
            return {
                "type": "LOOP_BREAK",
                "step": self._step,
                "modified_messages": messages + [
                    {"role": "user", "content": fix_msg}
                ],
                "description": f"Injected loop-break at step {self._step}"
            }

        # CHECK 2: Context decay
        if self._context_pct() > 70:
            self._anomalies.append({
                "type": "CONTEXT_DECAY", "step": self._step,
                "severity": "HIGH", "detail": {"pct": self._context_pct()}
            })
            # Trim old messages, keep system + last 10 messages
            trimmed = messages[-10:] if len(messages) > 12 else messages
            fix_msg = (
                f"[AGENTLENS] Context window is {self._context_pct():.0f}% full. "
                f"Before continuing, provide a brief summary of progress so far "
                f"and key decisions made. Then continue your task."
            )
            return {
                "type": "CONTEXT_COMPACTION",
                "step": self._step,
                "modified_messages": trimmed + [
                    {"role": "user", "content": fix_msg}
                ],
                "description": (
```

```
                f"Compacted context at step {self._step} "
                f"({self._context_pct():.0f}% → trimmed to last 10 msgs)"
            )
        }

    # CHECK 3: Error spiral
    if self._error_streak >= 3:
        self._anomalies.append({
            "type": "ERROR_SPIRAL", "step": self._step,
            "severity": "HIGH",
            "detail": {"consecutive": self._error_streak}
        })
        fix_msg = (
            f"[AGENTLENS] {self._error_streak} consecutive errors detected. "
            f"Your current approach is not working. STOP making incremental "
            f"fixes. Instead: (1) State the root cause of these errors. "
            f"(2) Propose a completely different approach. "
            f"(3) Only proceed once you have a new strategy."
        )
        return {
            "type": "ERROR_SPIRAL_BREAK",
            "step": self._step,
            "modified_messages": messages + [
                {"role": "user", "content": fix_msg}
            ],
            "description": (
                f"Broke error spiral at step {self._step} "
                f"({self._error_streak} consecutive errors)"
            )
        }

    return None

def _detect_loop(self):
    """Check if last N tool calls form a repeating pattern."""
    if len(self._tool_history) < 6:
        return None
    recent = self._tool_history[-12:]
    for pattern_len in range(2, 5):
        pattern = recent[-pattern_len:]
        count = 0
        for i in range(len(recent) - pattern_len, -1, -pattern_len):
            segment = recent[i:i + pattern_len]
            if segment == pattern:
```

```
                count += 1
            else:
                break
        if count >= 2:
            return {
                "pattern": " → ".join(t[0] for t in pattern),
                "count": count
            }
    return None

def _context_pct(self):
    total = self._cumulative_input + self._cumulative_output
    return (total / 200000) * 100

def _build_log_entry(self, kwargs, response, duration_ms):
    tool_calls = []
    text_content = ""
    has_error = False

    for block in response.content:
        if block.type == "tool_use":
            input_str = str(block.input)[:200]
            tool_calls.append({
                "name": block.name, "input_summary": input_str
            })
            self._tool_history.append(
                (block.name, hash(input_str) % 10000)
            )
        elif block.type == "text":
            text_content = block.text[:500]
            if "error" in block.text.lower()[:200]:
                has_error = True

    self._cumulative_input += response.usage.input_tokens
    self._cumulative_output += response.usage.output_tokens

    return {
        "step_id": self._step,
        "timestamp": datetime.utcnow().isoformat(),
        "input_tokens": response.usage.input_tokens,
        "output_tokens": response.usage.output_tokens,
        "cumulative_tokens": self._cumulative_input + self._cumulative_output,
        "context_utilization_pct": round(self._context_pct(), 1),
        "duration_ms": round(duration_ms),
```

```
        "stop_reason": response.stop_reason,
        "tools_called": tool_calls,
        "text_summary": text_content,
        "message_count": len(kwargs.get("messages", [])),
        "model": kwargs.get("model", "unknown"),
        "intervention_applied": (
            self._interventions[-1]["type"]
            if self._interventions
            and self._interventions[-1]["step"] == self._step
            else None
        ),
        "anomalies_at_step": [
            a for a in self._anomalies if a["step"] == self._step
        ],
        "health_score": self._compute_health(),
    }

def _update_tracking(self, response, log_entry):
    has_error = any(
        "error" in tc.get("input_summary", "").lower()
        for tc in log_entry["tools_called"]
    ) or "error" in log_entry.get("text_summary", "").lower()

    if has_error:
        self._error_streak += 1
    else:
        self._error_streak = 0

def _compute_health(self):
    severity_costs = {"CRITICAL": 20, "HIGH": 10, "MEDIUM": 5}
    penalty = sum(
        severity_costs.get(a["severity"], 0) for a in self._anomalies
    )
    return max(0, min(100, 100 - penalty))

# ── RECOVERY API ──

def find_recovery_point(self):
    """Find the last healthy checkpoint before anomalies started."""
    if not self._anomalies:
        return None
    first_anomaly = min(a["step"] for a in self._anomalies)
    recovery_step = max(1, first_anomaly - 2)
    path = self._checkpoint_dir / f"step_{recovery_step:04d}.json"
```

```
    if path.exists():
        with open(path) as f:
            return json.load(f)
    return None

def recover_with_fix(self, fix_strategy=None):
    """
    Load last healthy checkpoint and generate a corrected continuation.
    If fix_strategy is None, uses Claude to auto-generate one.
    """
    checkpoint = self.find_recovery_point()
    if not checkpoint:
        return None

    if fix_strategy is None:
        fix_strategy = self._auto_generate_fix()

    messages = checkpoint["messages"]
    messages.append({
        "role": "user",
        "content": (
            f"[RECOVERY] Your previous approach failed after this point. "
            f"Analysis: {fix_strategy} "
            f"Resume your task with this corrected approach."
        )
    })

    # Reset internal state
    self._step = checkpoint["step_id"]
    self._anomalies = []
    self._error_streak = 0
    self._tool_history = []
    self._interventions = []

    return {
        "recovery_step": checkpoint["step_id"],
        "messages": messages,
        "fix_strategy": fix_strategy,
    }

def _auto_generate_fix(self):
    """Use Claude to analyze failures and generate fix strategy."""
    anomaly_summary = json.dumps(self._anomalies[:10], indent=2)
    response = self._client.messages.create(
```

```
        model="claude-sonnet-4-5-20250929",
        max_tokens=300,
        messages=[{
            "role": "user",
            "content": (
                f"An AI coding agent failed with these anomalies:\n"
                f"{anomaly_summary}\n\n"
                f"In 2-3 sentences, explain what went wrong and give a "
                f"concrete, actionable correction the agent should follow "
                f"to avoid the same failure. Be specific."
            )
        }]
    )
    return response.content[0].text

# ── EXPORT FOR DASHBOARD ──

def export_analysis(self, output_path="analysis.json"):
    """Export full analysis for the web dashboard."""
    self._log_file.close()
    log_path = self._log_file.name
    steps = []
    with open(log_path) as f:
        for line in f:
            if line.strip():
                steps.append(json.loads(line))

    analysis = {
        "total_steps": len(steps),
        "health_score": self._compute_health(),
        "anomalies": self._anomalies,
        "interventions": self._interventions,
        "steps": steps,
        "checkpoints": sorted([
            str(p) for p in self._checkpoint_dir.glob("*.json")
        ]),
        "recovery_point": (
            self.find_recovery_point() or {}
        ).get("step_id"),
        "summary": {
            "total_tokens": steps[-1]["cumulative_tokens"] if steps else 0,
            "final_context_pct": (
                steps[-1]["context_utilization_pct"] if steps else 0
            ),
```

```
        "total_tool_calls": sum(
            len(s.get("tools_called", [])) for s in steps
        ),
        "interventions_made": len(self._interventions),
        "anomalies_detected": len(self._anomalies),
    }
}

with open(output_path, "w") as f:
    json.dump(analysis, f, indent=2)
return analysis
```

Test immediately with a 3-step API call to verify it works.

Deliverable: agentlens/interceptor.py — working SDK wrapper with logging, checkpoints, anomaly detection, and intervention

---

HOUR 0:45–1:30 — Demo Agent Script

Goal: Two trajectories — one without AgentLens, one with

Build `demo/run_unprotected.py`:

```python
"""Agent WITHOUT AgentLens — shows the failure"""
import anthropic

client = anthropic.Anthropic()
# Run a coding task that naturally triggers loops/context rot
# Save raw conversation to unprotected_trajectory.jsonl
```

Build `demo/run_protected.py`:

```python
"""Same agent WITH AgentLens — shows interventions saving it"""
from agentlens import AgentLens

client = AgentLens(mode="protect")
# Run the SAME coding task
# AgentLens automatically intervenes when failures start
# Save to protected_trajectory.jsonl + analysis.json
```

ALSO build `demo/generate_synthetic.py`: A script that generates two fake but realistic trajectory JSONs:

- `unprotected_sample.json` — 30 steps, degrading into failure
- `protected_sample.json` — 30 steps, with interventions keeping it healthy

This is your safety net. Build it TONIGHT if possible.

The synthetic trajectories should tell this story:

```
UNPROTECTED:
Step 1-8:  🟢 Healthy — reads files, makes plans, edits code
Step 9-12:  🟡 Degrading — re-reads files, context growing fast
Step 13-16: 🔴 Looping — read→write→test→read→write→test cycle
Step 17-22: 🔴 Error spiral — test fails, fix breaks, test fails again
Step 23-28: 🔴 Context rot — 85%+ utilization, repeating itself
Step 29-30: 💀 Dead — meaningless output, context exhausted

PROTECTED (same task):
Step 1-8:  🟢 Healthy — same as above
Step 9-12:  🟡 Degrading — same initial signs
Step 13:   🛡️ INTERVENTION: Loop detected → course correction injected
Step 14-18: 🟢 Recovered — agent takes new approach, back on track
Step 19:   🛡️ INTERVENTION: Context at 72% → compaction triggered
Step 20-25: 🟢 Healthy — context freed up, agent continues cleanly
Step 26-30: 🟢 Completes task successfully

Health score: Unprotected 12/100 → Protected 78/100
```

Deliverable: Two trajectory JSON files (synthetic or real) telling the A/B story

---

HOUR 1:30–2:00 — Recovery Module + Export Pipeline

Goal: Recovery API works, analysis.json exports correctly

- Test `find_recovery_point()` on unprotected trajectory
- Test `recover_with_fix()` generates a fix strategy
- Test `export_analysis()` produces clean JSON the dashboard can consume
- Structure the final analysis.json:

```
json
```

```
{
  "total_steps": 30,
  "health_score": 78,
  "mode": "protected",
  "anomalies": [
    {"type": "LOOP", "step": 13, "severity": "HIGH", "detail": {...}},
    {"type": "CONTEXT_DECAY", "step": 19, "severity": "HIGH", "detail": {...}}
  ],
  "interventions": [
    {
      "type": "LOOP_BREAK",
      "step": 13,
      "description": "Injected loop-break — agent was cycling read→write→test"
    },
    {
      "type": "CONTEXT_COMPACTION",
      "step": 19,
      "description": "Compacted context from 72% → 35% by trimming old messages"
    }
  ],
  "recovery_point": 11,
  "steps": [...],
  "summary": {
    "total_tokens": 156000,
    "final_context_pct": 42.3,
    "total_tool_calls": 67,
    "interventions_made": 2,
    "anomalies_detected": 2
  }
}
```

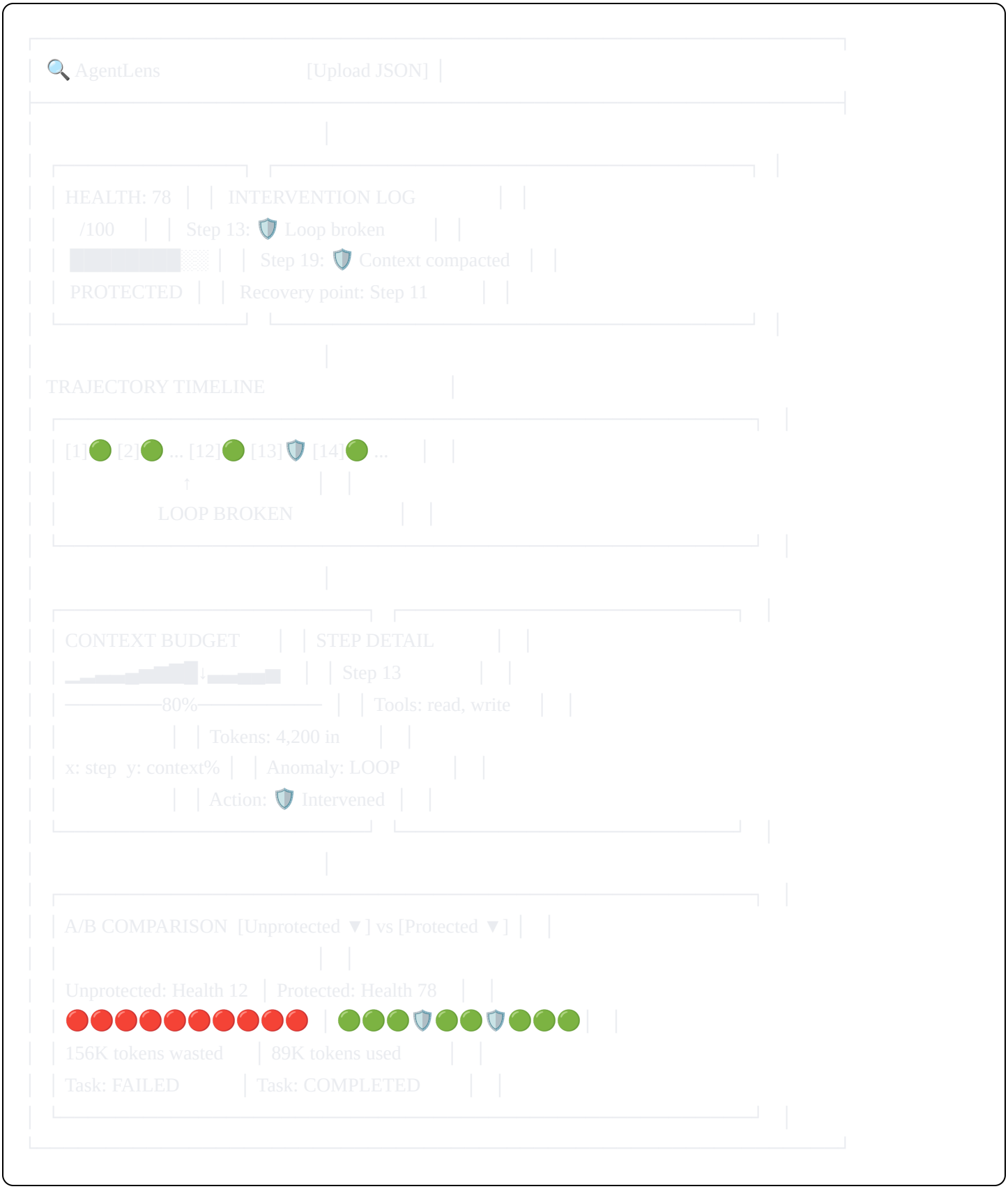

Deliverable: analysis.json ready for dashboard consumption

HOUR 2:00–4:30 — Web Dashboard (THE BIG ONE)

Goal: Fully functional, beautiful dashboard deployed on Replit

Design: Dark "mission control" with green/amber/red health indicators

Layout:



Build order (most impactful first):

1. **Timeline component** (45 min) — the hero visual
2. **Health gauge** (15 min) — big number that tells the story
3. **Context budget chart** (20 min) — Recharts AreaChart

4. **Intervention log** (15 min) — list of what AgentLens did
5. **Step detail panel** (20 min) — click-to-inspect
6. **A/B comparison view** (30 min) — side-by-side unprotected vs protected
7. **JSON upload** (15 min) — so judges can try their own data
8. **Polish** (30 min) — animations, glow effects, responsive

A/B Comparison is the KILLER feature for the demo. It tells the story visually: "Without AgentLens, your agent dies. With AgentLens, it lives."

Deliverable: Deployed React app on Replit

HOUR 4:30–5:15 — A/B Demo Data + Claude Meta-Analysis

Goal: Side-by-side comparison data + AI-generated narrative

- Generate or refine both trajectory JSONs (protected + unprotected)
- Run Claude meta-analysis on both trajectories
- Add narrative cards to dashboard:
  - "WITHOUT AgentLens: Agent entered a fix-break-fix loop at step 13..."
  - "WITH AgentLens: Loop detected at step 13, intervention injected..."

Deliverable: Dashboard with A/B comparison and AI narratives

HOUR 5:15–6:00 — Integration Test + End-to-End Verification

Goal: Everything works perfectly

Run the full pipeline:

- [ ] `python demo/run_protected.py` → trajectory.jsonl + checkpoints/
- [ ] `python -c "from agentlens import AgentLens; a = AgentLens(); print('OK')"`
- [ ] analysis.json loads in dashboard correctly
- [ ] Timeline renders all steps with correct colors
- [ ] Click a step → detail panel shows
- [ ] Click an intervention step → shows what AgentLens did
- [ ] A/B comparison renders both trajectories
- [ ] Health scores are correct
- [ ] Context chart looks right
- [ ] Recovery point is marked on timeline

- [ ] Dashboard is accessible via Replit public URL

Fix any broken things. Do NOT add new features.

HOUR 6:00–6:30 — README + Repo Cleanup

Write `README.md`:

markdown

```
# 🔍 AgentLens — An Immune System for AI Agents

AgentLens detects, prevents, and heals AI agent failures in real-time.

## The Problem
AI agents fail silently. They loop, contradict themselves, exhaust
context, and spiral into errors — and you have no visibility into why.

## The Solution
AgentLens wraps your Anthropic SDK client in 2 lines of code and:
- **Monitors** every tool call, token count, and decision
- **Detects** loops, context rot, error spirals, and repetition
- **Intervenes** automatically to course-correct before failures compound
- **Recovers** by rewinding to a healthy checkpoint with a fix applied
- **Visualizes** everything in a real-time dashboard

## Quick Start
\```python
# Before (unprotected)
client = anthropic.Anthropic()

# After (protected by AgentLens)
from agentlens import AgentLens
client = AgentLens(mode="protect")

# That's it. Everything else is automatic.
\```

## Dashboard
[Live demo on Replit →](YOUR_REPLIT_URL)

## Built at Build India 2026
Anthropic × Replit × Lightspeed Hackathon
```

HOUR 6:30–7:00 — Demo Rehearsal

Practice the 3-minute demo exactly:

0:00–0:30 — The Problem "Every team deploying AI agents hits the same wall." Show the unprotected trajectory — red timeline, health score 12, task failed.

0:30–1:30 — The Solution (AgentLens) "Two lines of code to protect any agent." Show the code diff (anthropic.Anthropic → AgentLens). Switch to protected trajectory — green timeline, health score 78, task completed. Point to the intervention markers: "Here at step 13, AgentLens detected a loop forming and automatically injected a course correction. The agent recovered without any human intervention."

1:30–2:15 — The Dashboard Click through: timeline, health gauge, context chart, step detail. Show the A/B comparison side by side. "Same task. Same model. One has AgentLens, one doesn't."

2:15–2:45 — Recovery "And if interventions aren't enough, AgentLens saves checkpoints at every step. You can rewind to the last healthy state, and it uses Claude to generate a fix strategy automatically." Show the recovery panel.

2:45–3:00 — The Pitch "This is the observability layer every AI agent needs. Open-source SDK, paid dashboard. Datadog was for servers. AgentLens is for agents."

---

9. WHAT TO SKIP

- ❌ Authentication / login
- ❌ Database (JSON files only)
- ❌ Websockets / real-time streaming
- ❌ pip packaging
- ❌ Tests
- ❌ Multi-provider support (Anthropic only for now)
- ❌ Multiple simultaneous sessions
- ❌ Custom anomaly detection rules
- ❌ Branching visualization (mention as roadmap)

---

10. PRE-HACKATHON PREP (DO TONIGHT)

- [ ] Generate synthetic unprotected_trajectory.json (30 steps, failing)
- [ ] Generate synthetic protected_trajectory.json (30 steps, with interventions)

- [ ] Test Anthropic API key works
- [ ] Set up Replit project (React template)
- [ ] Have this document open and printed
- [ ] Save Recharts docs offline / bookmarked
- [ ] Save Tailwind color reference
- [ ] Practice explaining the concept in 30 seconds
- [ ] Sleep.

---

11. PROJECT NAME AND BRANDING

Name: AgentLens

Tagline options:

- "An immune system for AI agents"
- "See exactly when and why your agent broke — then fix it automatically"
- "Detect. Prevent. Recover."

Logo concept (for dashboard header): 🔍 with a shield overlay — represents observability + protection

Color scheme:

- Background: #0a0a0f (near black)
- Surface: #141420 (dark card bg)
- Border: #1e1e30 (subtle borders)
- Text: #e2e8f0 (light gray)
- Green (healthy): #22c55e
- Amber (warning): #f59e0b
- Red (critical): #ef4444
- Blue (intervention): #3b82f6
- Shield/intervention icon: #8b5cf6 (purple — stands out from health colors)

Fonts:

- Headers: DM Sans (bold, modern)
- Data/metrics: JetBrains Mono (monospace, technical feel)